La librairie pandas : première approche

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Series/Dataframe : qu'est ce que c'est ? **4**

A. Introduction 4

B. Création d'une Series 5

C. Création d'un Dataframe 6

D. Exercice : Quiz 8

III - Manipulation d'un Dataframe **10**

A. Récupérer des données 10

B. Modifier le contenu 11

C. Opérations de base 12

D. Exercice : Quiz 13

IV - L'essentiel **15**

V - Auto-évaluation **16**

A. Exercice 16

B. Test 16

Solutions des exercices **18**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : bases en Python, connaissance de Numpy, bases en mathématiques (matrices)

Contexte

Pandas se base sur la librairie Numpy. Il s'agit d'une librairie qui permet de manipuler des tableaux de données (1D ou 2D). Pour ceux qui connaissent, Pandas est un peu comme Excel, mais en mieux !

Le développement de cette librairie a été optimisé de manière à pouvoir manipuler des (très) grands volumes de données très rapidement. C'est pour cette raison que Pandas devient quasiment incontournable lorsque les données deviennent importantes.

Pandas est utilisé en production dans de nombreux secteurs, comme par exemple la finance. Par ailleurs, l'objectif de l'équipe de développeurs de Pandas est qu'elle devienne la librairie open-source d'analyse et de manipulation de données la plus puissance et la plus flexible qui soit, tous langages confondus, rien que ca !

Dans ce cours, nous verrons les deux types de données fondamentaux qui sont manipulés avec Pandas : la Series et le Dataframe. Nous verrons comment les construire et comment les utiliser efficacement.

Tous droits réservés à STUDI - Reproduction interdite

II Series/Dataframe : qu'est ce que c'est ?

A. Introduction

Series — Az Définition

Une Series n'est rien de plus qu'un vecteur de données. Il s'agit d'un tableau à une seule dimension. Chaque élément de la Series est associé à un index qui démarre à 0.

Indices dans les tableaux — Rappel

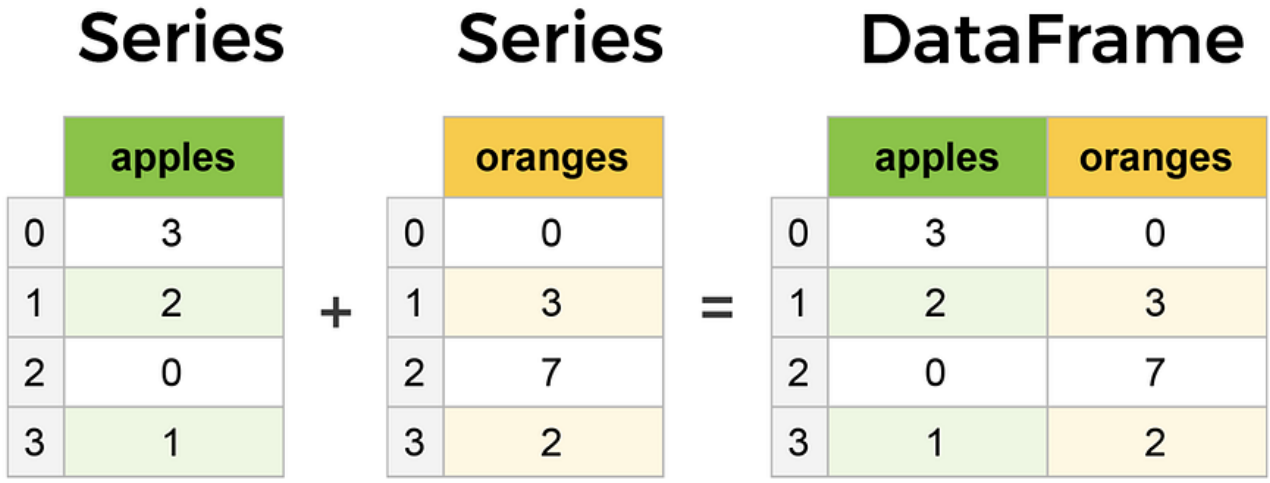
Rappelez-vous lorsque nous manipulions des données avec la librairie Numpy (np.array, etc.), nous avions déjà mis en évidence le fait que les indices dans les tableaux démarraient à 0. Nous verrons que pour les objets Series et Dataframes, c'est aussi le cas.

Dataframe — Az Définition

Un Dataframe est formé d'une ou de plusieurs Series. S'il contient au moins deux Series, il devient donc un tableau à deux dimensions. Les Series qui le constituent sont les colonnes de ce tableau. Chaque ligne est, à l'instar des tableaux Numpy ou des Series, indexée par un entier à partir de 0.

Series et Dataframe

Pour illustrer la Series et le Dataframe, voici un simple exemple visuel, qui montre comment un Dataframe est créé à partir de deux Series :



Source : Meduim - Introduction to Pandas Series and DataFrame[1]

1. https://medium.com/@debopamdeycse19/introduction-to-pandas-for-data-science-d5a575e4cd9b

Tous droits réservés à STUDI - Reproduction interdite

B. Création d'une Series

Pour construire une Series, on utilise la fonction Series() de Pandas en lui passant en argument une liste de données. Voyons ceci dans l'exemple suivant :

Series Exemple

Supposons qu'on souhaite définir une Series qui reprend les noms de quelques planètes du système solaire :

```
import pandas as pd

names = pd.Series(["Jupiter", "Saturn", "Uranus", "Neptune", "Earth"])

print(names)
```

Ce qui nous renvoie :

```
0    Jupiter
1     Saturn
2     Uranus
3    Neptune
4      Earth
dtype: object
```

On voit qu'il s'agit bien d'une « *liste* » de noms, chacun étant associé à un index démarrant à 0.

Le type d'objets qui constitue notre Series est « *object* ». Il s'agit simplement dans notre cas de chaines de caractères.

Les Series possèdent des attributs qui nous permettent de récupérer, par exemple, la liste des index ou encore le contenu « *brut* » de celles-ci :

Series Exemple

```
import pandas as pd

names = pd.Series(["Jupiter", "Saturn", "Uranus", "Neptune", "Earth"])

print(f"Raw data : {names.values}")
print(f"Index : {names.index}")
```

Ce qui nous renvoie :

```
Raw data : ['Jupiter' 'Saturn' 'Uranus' 'Neptune' 'Earth']
Index : RangeIndex(start=0, stop=5, step=1)
```

Les données brutes sont simplement le contenu de notre Series, tandis que l'index nous informe que le tableau contient 6 éléments (indexés de 0 à 5), et que chaque index est séparé d'une unité.

Tous droits réservés à STUDI - Reproduction interdite

C. Création d'un Dataframe

Nous avons vu qu'un Dataframe n'est rien de plus qu'un « *empilement horizontal* » de Series (les Series sont les colonnes de notre Dataframe).

Il y a énormément de façons de créer un Dataframe. Dans le cadre de ce cours, nous verrons les méthodes suivantes :

- Par concaténation horizontale de Series
- Par création d'un tableau Numpy
- Par import d'un fichier Excel ou d'un fichier CSV

Concaténation horizontale de Series

Exemple

Reprenons l'exemple des planètes. Nous allons ajouter une deuxième Series, qui contiendra le diamètre de chacune des planètes constituant notre liste. Ensuite nous ajouterons un nom à chaque Series, et nous les concatènerons pour en faire un dataframe :

```
import pandas as pd

names = pd.Series(      ["Jupiter",   "Saturn",   "Uranus",   "Neptune",
"Earth"])
diameters = pd.Series(  [142984,      120536,     51118,      49528,
12756])

names.name = "Planets"
diameters.name = "Diameters (km)"
df = pd.concat([names, diameters], axis=1)

print(df)
```

Ce qui nous renvoie :

```
   Planets  Diameters
0  Jupiter     142984
1   Saturn     120536
2   Uranus      51118
3  Neptune      49528
4    Earth      12756
```

On voit que les Series sont bien nos colonnes. Spécifier l'axe de concatenation (axis) permet de concaténer les séries dans le sens horizontal (c'est-à-dire que nos Series seront nos colonnes).

Dataframe depuis un tableau Numpy

Exemple

Comme nous l'avons vu précédemment dans cette formation, Numpy nous permet de créer des tableaux très simplement. Dans cet exemple, nous allons créer un Dataframe en utilisant des tableaux (arrays) Numpy :

```
import pandas as pd
import numpy as np

myNumpyArray = np.array([ [1, 2], [3, 4], [5, 6]])


df = pd.DataFrame(myNumpyArray, columns=list(["Col 1", "Col 2"]))

print(df)
```


Tous droits réservés à STUDI - Reproduction interdite

Dans cet exemple, on voit qu'on peut spécifier directement une liste de noms de colonnes lors de la création du Dataframe.

Ce code nous renvoie ceci :

```
   Col 1  Col 2
0      1      2
1      3      4
2      5      6

```

Ce qui correspond bien à notre tableau Numpy.

Dataframe par import Excel

Exemple

Supposons que nous disposons d'un tableau Excel qui contient les éléments suivants :

	A	B
1	Planets	Diameters
2	Jupiter	142984
3	Saturn	120536
4	Uranus	51118
5	Neptune	49528
6	Earth	12756

Supposons que ce fichier a le nom « *myTable.xlsx* ».

Voici comment on peut importer son contenu pour en faire un Dataframe :

```python
import pandas as pd
import numpy as np

df = pd.read_excel('myTable.xlsx')

print(df)
```

Ce qui nous renvoie :

```
   Planets  Diameters
0  Jupiter     142984
1   Saturn     120536
2   Uranus      51118
3  Neptune      49528
4    Earth      12756
```

Import Excel

Attention

Importer un fichier Excel ne peut pas se faire sans la dépendance openpyxl. N'oubliez donc pas, avant d'exécuter l'exemple précédent, d'installer openpyxl. Si vous utilisez conda, voici la commande à exécuter dans votre environnement virtuel :

```
conda install openpyxl
```

Tous droits réservés à STUDI - Reproduction interdite

À l'instar des Series, les Dataframe contiennent plusieurs attributs très utles, qui permettent de récuperer la liste des index, la liste des noms de colonnes, la taille du Dataframe, ou même l'empreinte mémoire de celui-ci. Voyons ceci dans l'exemple ci-dessous :

Exemple

Reprenons l'exemple précédent, pour lequel notre Dataframe était le suivant :

```
> print(df)
  Planets  Diameters
0  Jupiter     142984
1   Saturn     120536
2   Uranus      51118
3  Neptune      49528
4    Earth      12756
```

```
print(f"Index : {df.index}")
print(f"Columns : {df.columns}")
print(f"Shape : {df.shape}")
print(f"Memory usage : {df.memory_usage()}")
```

Qui nous renvoie le résultat suivant :

```
Index   : RangeIndex(start=0, stop=5, step=1)
Columns : Index(['Planets', 'Diameters'], dtype='object')
Shape   : (5, 2)
Memory usage :   Index        132
                Planets       40
                Diameters     40
                dtype: int64
```

On remarque que memory_usage renvoie l'utilisation mémoire pour chaque colonne séparément. L'unité est en Bytes. Il est également possible de récupérer l'occupation mémoire totale du Dataframe en sommant le résulat : `sum(df.memory_usage)`.

D. Exercice : Quiz

[solution n°1 p. 18]

Question 1

Une Series, c'est :

- Une suite de nombres qui évoluent dans un ordre croissant
- Un tableau à 2 dimensions
- Un vecteur de nombres (tableau à une seule dimension)

Question 2

Un Dataframe, c'est :

- Un graphique qui contient des données numériques
- Un vecteur de données numériques

Tous droits réservés à STUDI - Reproduction interdite

Un tableau à 2 dimensions qui contient des objets

Question 3

Pour donner un nom à ma Series, j'écrirai :

mySeries.name = 'mon Nom'

mySeries.name(« *mon Nom* »)

On ne peut pas donner de nom à une Series, il s'agit uniquement de données numériques

Question 4

Contrairement aux objets « *Array* » de la librairie Numpy, il n'est pas possible de récupérer la taille (x,y) d'un Dataframe :

Vrai

Faux

Il faut d'abord convertir le Dataframe en Array numpy pour pouvoir le faire

Question 5

La méthode memory_usage() d'un Dataframe renvoie :

L'empreinte mémoire (en Bytes) de chaque colonne de notre dataframe

Le nombre d'éléments de chaque colonne de notre dataframe

Le nombre total d'éléments qui constituent notre dataframe

Tous droits réservés à STUDI - Reproduction interdite

III Manipulation d'un Dataframe

A. Récupérer des données

Pour récupérer les éléments d'un Dataframe, il existe plusieurs solutions. Les crochets `[]` permettent de récupérer une colonne spécifique d'un Dataframe. Les deux points « **:** » permettent de récupérer un ensemble de lignes, tandis que les méthodes `loc()` et `iloc()` permettent de récupérer certaines lignes sur base de leur label ou de leur index.

Récupérer les données dans un Dataframe

Exemple

Sur base de l'exemple précédent, nous disposons du Dataframe suivant :

```
  Planets  Diameters
0 Jupiter     142984
1  Saturn     120536
2  Uranus      51118
3 Neptune      49528
4   Earth      12756
```

Nous pouvons récupérer son contenu comme expliqué précédemment :

```
print(df["Planets"])
print(df[1:3])
print(df.loc[2, "Diameters"])
print(df.iloc[2,1])
```

Ce qui nous renvoie :

```
0    Jupiter
1     Saturn
2     Uranus
3    Neptune
4      Earth
Name: Planets, dtype: object

  Planets  Diameters
1  Saturn     120536
2  Uranus      51118

51118

51118
```

Remarque

Il est également possible de ne filtrer la sélection que sur les colonnes ou sur les lignes. Ceci peut se faire en précisant qu'on conserve toutes les lignes ou toutes les colonnes, via les deux points « : »

Tous droits réservés à STUDI - Reproduction interdite

Initialisation de matrices Exemple

Si on veut récupérer toutes les lignes de la colonne « *Diameters* », on peut également écrire :

```
print(df.loc[ :, "Diameters"])
```

Ceci nous renverra :

```
0    142984
1    120536
2     51118
3     49528
4     12756
Name: Diameters, dtype: int64
```

Parfois, on souhaite filtrer le contenu selon une condition, par exemple, si on souhaite récupérer uniquement les données du Dataframe pour lesquelles le diamètre est supérieur à 120.000 km :

Initialisation de matrices Exemple

```
print(df[df["Diameters"] > 120000])
```

Ceci nous renverra :

```
   Planets  Diameters
0  Jupiter     142984
1   Saturn     120536
```

Les autres planètes ne répondant pas au critère de la sélection, elles ne sont pas affichées.

B. Modifier le contenu

Pour modifier le contenu d'un Dataframe, on peut utiliser les méthodes at() et iat(), comme illustré dans l'exemple ci-dessous :

at() et iat() Exemple

Sur base de l'exemple précédent, nous disposons du Dataframe suivant :

```
   Planets  Diameters
0  Jupiter     142984
1   Saturn     120536
2   Uranus      51118
3  Neptune      49528
4    Earth      12756
```

Supposons que nous souhaitions modifier le diamètre de la terre :

```
df.at[4,"Diameters"] = 13000
```

Nous obtenons ainsi le Dataframe suivant :

```
   Planets  Diameters
0  Jupiter     142984
1   Saturn     120536
2   Uranus      51118
3  Neptune      49528
4    Earth      13000
```

On voit que le diamètre de la terre a bien été modifié.

Tous droits réservés à STUDI - Reproduction interdite

Ceci est équivalent à utiliser `iat()`, mais en spécifiant l'index et non le titre de la colonne :

```
df.iat[4,1] = 13000
```

C. Opérations de base

Nous allons voir quelques opérations de base sur les Dataframe qui permettent de générer quelques statistiques :

Opérations de base

Exemple

Nous repartons à nouveau de l'exemple précédent :

```
   Planets  Diameters
0  Jupiter     142984
1   Saturn     120536
2   Uranus      51118
3  Neptune      49528
4    Earth      12756

```

Nous pouvons facilement afficher quelques informations sur nos données, par exemple des données statiques, le nombre de données ou encore afficher ces données sous forme de graphique :

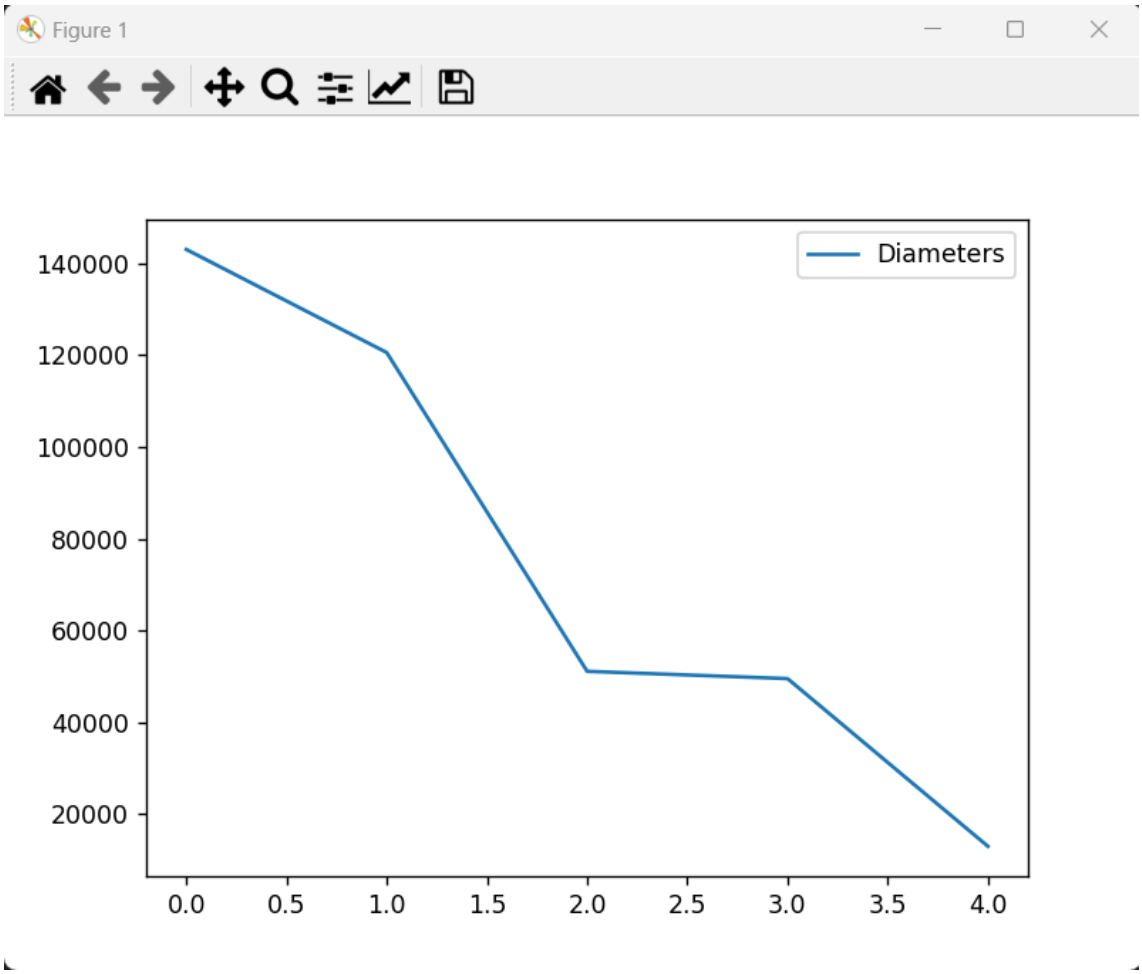
```
print(df.describe())
print(df["Diameters"].mean())
print(df.value_counts())
df.plot()
plt.show()
```

Ce qui nous renvoie par exemple :

```
           Diameters
count       5.000000
mean    75433.200000
std     54216.201335
min     13000.000000
25%     49528.000000
50%     51118.000000
75%    120536.000000
max    142984.000000

75433.2

Planets  Diameters
Earth    13000        1
Jupiter  142984       1
Neptune  49528        1
Saturn   120536       1
Uranus   51118        1
Name: count, dtype: int64
```


Tous droits réservés à STUDI - Reproduction interdite

L'appel à plt.show() nous permet d'afficher notre graphique :



Notez que dans cet exemple, la fonction value_count() renvoie 1 pour chacun des éléments, puisque chaque planète de notre tableau a un diamètre différent. Cette fonction permet en effet de récupérer la fréquence d'occurrence des lignes identiques.

D. Exercice : Quiz

[solution n°2 p. 19]

Question 1

Pour récupérer l'élément à la 1ere ligne, 3e colonne d'un Dataframe « *df* », on peut écrire :

- df.iloc[1,3]
- df.loc[1,3]
- df[1,3]

Question 2

Le code suivant nous affiche… :

```
print(df["Diameters"] <= 10500)
```

- Tous les éléments d'un Dataframe « *df* »

Tous droits réservés à STUDI - Reproduction interdite

Ce code provoque une erreur

Ce code affiche les éléments dont le diamètre est inférieur ou égal à 10500

Question 3

Pour modifier un élément à la position (i,j) d'un Dataframe, je peux écrire :

df.iloc[i,j] = …

df.iat[i,j] = …

df.iat(i,j) = …

Question 4

Le code suivant :

```
print(df.value_counts())
```

Affiche :

Le nombre total d'éléments du tableau

Le nombre de lignes identiques dans le tableau

Le nombre d'élément par colonne

Question 5

Pour afficher un graphique de mon Dataframe, je peux écrire :

df.plot()

On ne peut pas le faire directement, il faut transformer le Dataframe

df.graph()

Tous droits réservés à STUDI - Reproduction interdite

IV L'essentiel

Les éléments fondamentaux de la librairie Pandas sont les *Series* et les *Dataframes*. Dans ce cours, nous avons appris ce qu'était chacun de ces objets, et nous avons appris à les utiliser de façon simple et efficace.

Pandas est une librairie indispensable dès que des données numériques sont manipulées. Il est souvent comparé, fonctionnellement, à un tableur comme Excel, mais il permet de faire bien plus de choses et surtout de manière beaucoup plus rapide, grâce à son utilisation sous Python.

Ainsi, la manipulation de grandes quantités de données numériques devient très simple et rapide.

Dans un prochain cours, nous verrons quelques fonctions avancées de Pandas. Vous verrez qu'il s'agit d'une librairie extrêmement riche en fonctionnalités.

Tous droits réservés à STUDI - Reproduction interdite

V Auto-évaluation

A. Exercice

Vous êtes administrateur d'un site web. Vous avez lancé un sondage aux visiteurs de votre site, et voici les résultats que vous avez obtenus :

Visiteur	Date de visite	Durée (sec)	Nombre de clicks
1	20-06-2024	180	27
2	25-07-2024	200	34
3	21-06-2024	10	2
4	21-06-2024	300	42
5	22-06-2024	650	80
6	18-06-2024	120	15
7	10-05-2024	60	10
8	11-06-2024	2	1
9	13-06-2024	89	65
10	02-05-2024	110	80

Question 1

[solution n°3 p. 22]

Intégrez ces données dans un Dataframe, soit en les définissant par un tableau, soit en passant par un tableur externe (Excel, Calc, etc.) ;

Question 2

[solution n°4 p. 22]

Certaines de ces données trompent nos statistiques. En effet, certains visiteurs ont des durées de visite très courtes, ce qui signifie probablement qu'ils sont arrivés sur notre site par erreur.

Filtrez les résultats afin de supprimer les visites dont la durée est inférieure à 30 secondes.

Question 3

[solution n°5 p. 23]

Calculez la moyenne de la durée des visites ainsi que le nombre moyen de clicks obtenus parmi les résultats filtrés. Ensuite, tracez un graphe de la durée de visite en fonction de la date.

B. Test

Exercice 1 : Quiz

[solution n°6 p. 24]

Question 1

La librairie Pandas sert à … :

- Manipuler des tableaux à N dimensions
- Manipuler des nombres et tracer des graphiques

Tous droits réservés à STUDI - Reproduction interdite

Manipuler des vecteurs ou des tableaux à 2 dimensions efficacement

Question 2

La différence entre un Dataframe et une Series est :

Les méthodes qu'on peut appliquer

C'est exactement la même chose

Le Dataframe est un empilement horizontal de Series

Question 3

Il est possible de créer un Dataframe en l'important d'Excel :

Non, il faut le taper élément par élément

Oui, car Python peut lire des fichiers Excel grâce à Pandas

Oui, mais il est parfois nécessaire d'importer une librairie spécifique

Question 4

Pandas est très utile pour :

A Nettoyer des données dans un tableau

B Calculer toutes sortes d'indicateurs statistiques de nos données

C Faire des graphiques

D Calculer des fonctions mathématiques complexes

Question 5

Pandas est une librairie concurrente de Numpy :

Faux

Vrai

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 8]

Question 1

Une Series, c'est :

- Une suite de nombres qui évoluent dans un ordre croissant
- Un tableau à 2 dimensions
- ✓ Un vecteur de nombres (tableau à une seule dimension)

Une Series c'est un objet qui représente un vecteur de données. Chaque élément du vecteur est identifié par un index.

Question 2

Un Dataframe, c'est :

- Un graphique qui contient des données numériques
- Un vecteur de données numériques
- ✓ Un tableau à 2 dimensions qui contient des objets

Un Dataframe est un tableau à 2 dimensions qui contient des objets (par exemple, des données numériques). Chaque colonne du Dataframe est un objet Series.

Question 3

Pour donner un nom à ma Series, j'écrirai :

- ✓ mySeries.name = 'mon Nom'
- mySeries.name(« *mon Nom* »)
- On ne peut pas donner de nom à une Series, il s'agit uniquement de données numériques

Tous droits réservés à STUDI - Reproduction interdite

mySeries.name = « … » permet de donner un nom à l'attribut « *name* » de notre objet Series.

Question 4

Contrairement aux objets « *Array* » de la librairie Numpy, il n'est pas possible de récupérer la taille (x,y) d'un Dataframe :

- Vrai
- ✓ Faux
- Il faut d'abord convertir le Dataframe en Array numpy pour pouvoir le faire

La taille (nombre d'éléments lignes et colonnes) peut être simplement récupérée grâce à l'attribut shape : « *myDataframe.shape* ». C'est donc tout à fait possible

Question 5

La méthode memory_usage() d'un Dataframe renvoie :

- ✓ L'empreinte mémoire (en Bytes) de chaque colonne de notre dataframe
- Le nombre d'éléments de chaque colonne de notre dataframe
- Le nombre total d'éléments qui constituent notre dataframe

La fonction memory_usage() renvoie l'empreinte mémoire occupée par chacune des colonnes de notre Dataframe. L'unité est le Byte.

Solution n°2

[exercice p. 13]

Question 1

Pour récupérer l'élément à la 1ere ligne, 3e colonne d'un Dataframe « *df* », on peut écrire :

- ✓ df.iloc[1,3]
- df.loc[1,3]
- df[1,3]

Tous droits réservés à STUDI - Reproduction interdite

Pour récupérer les éléments par leur index, il faut utiliser « *iloc[]* ». loc[] peut être utilisée si l'on veut récupérer un élément en fonction de l'index de la ligne et du nom de la colonne correspondante. La troisième proposition n'est pas compatible avec les dataframes.

Question 2

Le code suivant nous affiche… :

```
print(df["Diameters"] <= 10500)
```

- Tous les éléments d'un Dataframe « *df* »
- Ce code provoque une erreur
- ✓ Ce code affiche les éléments dont le diamètre est inférieur ou égal à 10500

Ce code affiche en effet les éléments du Dataframe dont le diamètre est inférieur ou égal à 10500. Si nous avions voulu effectuer un filtrage sur les éléments, il aurait fallu écrire :

```
print(df[df["Diameters"] <= 10500])
```

Question 3

Pour modifier un élément à la position (i,j) d'un Dataframe, je peux écrire :

- df.iloc[i,j] = …
- ✓ df.iat[i,j] = …
- df.iat(i,j) = …

Il faut sélectionner l'élément [i,j] avec la fonction « *iat[]* ». La fonction « *iloc[]* » permet de récupérer le contenu d'un élément mais pas de le modifier directement.

Question 4

Le code suivant :

```
print(df.value_counts())
```

Affiche :

- Le nombre total d'éléments du tableau
- ✓ Le nombre de lignes identiques dans le tableau

Tous droits réservés à STUDI - Reproduction interdite

Le nombre d'élément par colonne

« *value_counts()* » identifie les lignes identiques et renvoie leur occurrence. À noter, le résultat de « *value_counts()* » est une Series.

Question 5

Pour afficher un graphique de mon Dataframe, je peux écrire :

✓ df.plot()

On ne peut pas le faire directement, il faut transformer le Dataframe

df.graph()

« *df.plot()* », généralement suivi de « *plt.show()* » permet d'afficher un graphique de notre dataframe. Il y a plusieurs options qui permettent de personnaliser l'affichage, que nous verrons dans un cours dédié aux notions avancées de Pandas.

Tous droits réservés à STUDI - Reproduction interdite

studi

[exercice p. 16] Solution n°3

Une façon de s'y prendre est d'entrer les données dans un tableur (par exemple, Excel) et d'enregistrer le document sous format .CSV.

Le contenu du fichier .CSV devrait ressembler à ceci :

```
Visiteur,Date,Duree,NombreClicks
1,20-06-24,180,27
2,25-07-24,200,34
3,21-06-24,10,2
4,21-06-24,300,42
5,22-06-24,650,80
6,18-06-24,120,15
7,10-05-24,60,10
8,11-06-24,2,1
9,13-06-24,89,65
10,02-05-24,110,80
```

L'import est alors immédiat dans notre application Python :

```python
import pandas as pd


df = pd.read_csv("stats_site.csv")
```

[exercice p. 16] Solution n°4

Pour ce faire, il suffit d'appliquer un filtre sur la colonne « *Durée* ». Puisque nos durées sont déjà exprimées en secondes, c'est immédiat :

```python
df_filtered = df[df['Duree']>30]
print(df_filtered)
```

Lorsque vous imprimez le contenu du dataframe, vous devriez obtenir uniquement les lignes pour lesquelles la durée de visite est supérieure à 30 secondes :

```
   Visiteur      Date  Duree  NombreClicks
0         1  20-06-24    180            27
1         2  25-07-24    200            34
3         4  21-06-24    300            42
4         5  22-06-24    650            80
5         6  18-06-24    120            15
6         7  10-05-24     60            10
8         9  13-06-24     89            65
9        10  02-05-24    110            80
```


Tous droits réservés à STUDI - Reproduction interdite

[exercice p. 16]

Solution n°5

La méthode describe() peut tout simplement nous aider à afficher la moyenne par colonne :

```
print(df_filtered.describe())
```

Ceci devrait vous donner le résultat suivant :

```
        Visiteur  Duree           NombreClicks
count   8.000000    8.000000       8.000000
mean    5.500000  213.625000      44.125000
std     3.162278  191.921068      27.833369
min     1.000000   60.000000      10.000000
25%     3.500000  104.750000      24.000000
50%     5.500000  150.000000      38.000000
75%     7.500000  225.000000      68.750000
max    10.000000  650.000000      80.000000

```

Ce qui répond à notre question : la moyenne de durée est de 213 secondes, tandis que le nombre de clicks moyen est de 44.

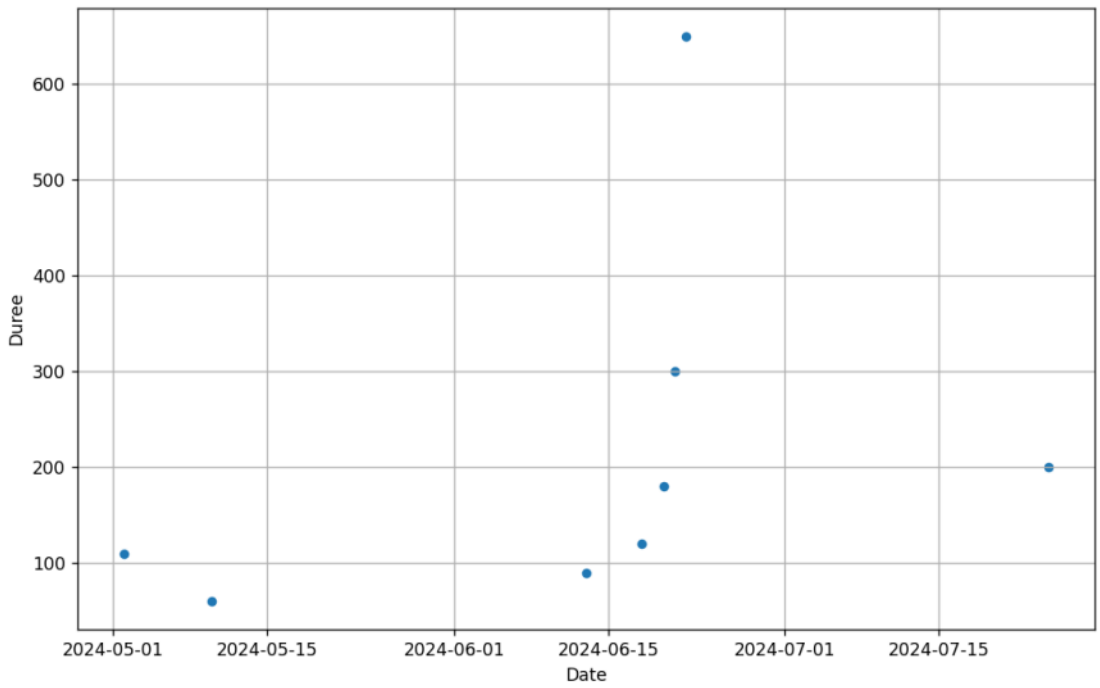
Traçons maintenant la durée de visite en fonction de la date. Attention, les dates doivent d'abord être converties en objets « *datetime* ». En effet, pour l'instant, le contenu de la colonne 'Date' n'est que du texte (string), Python ne peut donc pas réellement l'interpréter :

```
df_filtered2 = pd.DataFrame(df_filtered)
df_filtered2['Date'] = pd.to_datetime(df_filtered['Date'], format='%d-%m-%y')

print(df_filtered2)

df_filtered2.plot.scatter(x = 'Date', y = 'Duree')
plt.grid(True)
plt.show()
```

On construit d'abord un nouveau Dataframe (df_filtered2), qui est identique à df_filtered, sauf que sa colonne 'Date' contient des objets de type 'datetime'.

Ensuite, on trace simplement un graphique (nuage de points : scatter), en précisant les colonnes à utiliser pour les axes x et y.

Vous devriez alors obtenir le graphique suivant :



Tous droits réservés à STUDI - Reproduction interdite

studi

Solution n°6

[exercice p. 16]

Question 1

La librairie Pandas sert à … :

- Manipuler des tableaux à N dimensions
- Manipuler des nombres et tracer des graphiques
- ✓ Manipuler des vecteurs ou des tableaux à 2 dimensions efficacement

Pandas permet de manipuler des Series ou des Dataframes de manière optimisée. C'est une librairie qui propose énormément de fonctionnalités et qui peut traiter de très grand volumes de données.

Question 2

La différence entre un Dataframe et une Series est :

- Les méthodes qu'on peut appliquer
- C'est exactement la même chose
- ✓ Le Dataframe est un empilement horizontal de Series

Un Dataframe n'est rien d'autre qu'un empilement horizontal (en colonnes) d'éléments de type Series. Il est d'ailleurs possible de créer un Dataframe de cette façon.

Question 3

Il est possible de créer un Dataframe en l'important d'Excel :

- Non, il faut le taper élément par élément
- Oui, car Python peut lire des fichiers Excel grâce à Pandas
- ✓ Oui, mais il est parfois nécessaire d'importer une librairie spécifique

Python peut lire n'importe quel type de fichier Excel, à partir du moment où on installe la librairie openpyxl, qui permet effectivement de lire des fichiers Excel dans Python. La fonction de Pandas à utiliser ensuite est read_excel(). Cependant, il est possible d'importer des fichiers .CSV nativement à l'aide de la fonction read_csv().

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Pandas est très utile pour :

- **A** Nettoyer des données dans un tableau
- **B** Calculer toutes sortes d'indicateurs statistiques de nos données
- **C** Faire des graphiques
- **D** Calculer des fonctions mathématiques complexes

Bien que Pandas puisse être utilisé pour générer des graphiques, ses principaux atouts sont dans sa capacité à traiter des données d'entrée pour les filtrer, les nettoyer et en sortir des indicateurs. C'est une librairie extrêmement utile dans le domaine du Big Data ou des statistiques.

Question 5

Pandas est une librairie concurrente de Numpy :

- ✓ Faux
- Vrai

Pandas est une librairie qui se base sur Numpy, c'est-à-dire qu'elle utilise certaines fonctionnalités de Numpy en backend. De manière générale, Pandas est utilisée pour traiter des données tabulées, et les fonctions spécifiques de Numpy sont ensuite utilisées pour appliquer des opérations mathématiques plus complexe sur un sous-ensemble des données.

Tous droits réservés à STUDI - Reproduction interdite